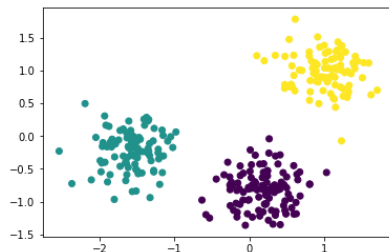


4A K MEAN CLUSTERING

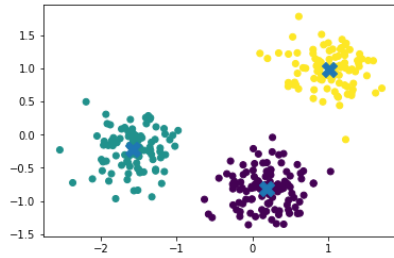
```
import pandas as pd
import numpy as np
df = pd.read_excel('ch1ex1.xlsx')
points = df.values
points
```

```
from sklearn.cluster import KMeans
model = KMeans(n_clusters=3)
model.fit(points)
labels = model.predict(points)
```

```
import matplotlib.pyplot as plt
xs = points[:,0]
ys = points[:,1]
plt.scatter(xs, ys, c=labels)
plt.show()
```



```
centroids = model.cluster_centers_
centroids_x = centroids[:,0]
centroids_y = centroids[:,1]
plt.scatter(xs, ys, c=labels)
plt.scatter(centroids_x, centroids_y, marker='X', s=200)
plt.show()
```

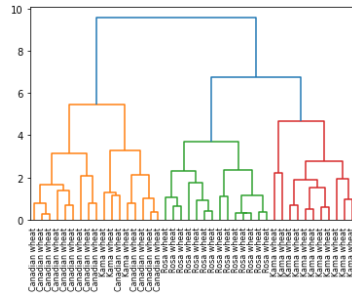


4B HIERARCHICAL CLUSTERING

```
import pandas as pd
seeds_df = pd.read_excel('seeds-less-rows.xlsx')
varieties = list(seeds_df.pop('grain_variety'))
samples = seeds_df.values

from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt
mergings = linkage(samples, method='complete')
dendrogram(mergings,
            labels=varieties,
            leaf_rotation=90,
            leaf_font_size=8,
)
plt.show()
```

```
from scipy.cluster.hierarchy import fcluster
labels = fcluster(mergings, 6, criterion='distance')
df = pd.DataFrame({'labels': labels, 'varieties': varieties})
ct = pd.crosstab(df['labels'], df['varieties'])
ct
```



varieties	Canadian wheat	Kama wheat	Rosa wheat
labels			
1	14	3	0
2	0	0	14
3	0	11	0

5A:-

Experiment: Usage of Sigmoid Activation Function in Artificial Neural Network

Aim:

To implement a perceptron model using the Sigmoid activation function for simulating basic logic gates like AND and NAND.

```
1]: import numpy as np
from functools import reduce

def perceptron(weight, bias, x):
    model = np.add(np.dot(x, weight), bias)
    print('model: {}'.format(model))
    logit = 1 / (1 + np.exp(-model))
    print('Type: {}'.format(logit))
    return np.round(logit)

def compute(logictype, weightdict, dataset):
    weights = np.array([weightdict[logictype][w] for w in weightdict[logictype].keys()])
    output = np.array([perceptron(weights, weightdict['bias'][logictype], val) for val in dataset])
    print(logictype)
    return logictype, output

def main():
    logic = {
        'logic_and': {'w0': -0.1, 'w1': 0.2, 'w2': 0.2},
        'logic_nand': {'w0': 0.6, 'w1': -0.8, 'w2': -0.8},
        'bias': {'logic_and': -0.2, 'logic_nand': 0.3}
    }

    dataset = np.array([
        [1, 0, 0],
        [1, 0, 1],
        [1, 1, 0],
        [1, 1, 1]
    ])

    logic_and = compute('logic_and', logic, dataset)
    logic_nand = compute('logic_nand', logic, dataset)

    def template(dataset, name, data):
```

```

logic_and = compute('logic_and', logic, dataset)
logic_nand = compute('logic_nand', logic, dataset)

def template(dataset, name, data):
    print("Logic Function: {}".format(name[6:].upper()))
    print("X0\tX1\tX2\tY")
    toPrint = ["{1}\t{2}\t{3}\t{0}".format(output, *datas) for datas, output in zip(dataset, data)]
    for i in toPrint:
        print(i)
    gates = [logic_and, logic_nand]
    for i in gates:
        template(dataset, *i)

if __name__ == '__main__':
    main()

```

```

model: -0.30000000000000004
Type: 0.425557483188341
model: -0.1
Type: 0.47502081252106
model: -0.1
Type: 0.47502081252106
model: 0.10000000000000003
Type: 0.52497918747894
logic_and
model: 0.8999999999999999
Type: 0.7109495026250039
model: 0.09999999999999992
Type: 0.5249791874789399
model: 0.09999999999999992
Type: 0.5249791874789399
model: -0.7
Type: 0.22104122770210220

```

5B FACE DETECTION

1 1 1 0.0

```
[16]: !pip install opencv-python
```

Requirement already satisfied: opencv-python in c:\users\lenovo\anaconda3\lib\site-packages
Requirement already satisfied: numpy<2.3.0,>=2 in c:\users\lenovo\anaconda3\lib\site-packag

```
[19]: import numpy as np
import cv2
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
img = cv2.imread('people.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.1, 5)
for (x,y,w,h) in faces:
    cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
    roi_gray = gray[y:y+h, x:x+w]
    roi_color = img[y:y+h, x:x+w]
cv2.imshow('img',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

6 TENSOR FLOWW

```
[26]: # Neural Network for Pima Indians Diabetes Prediction
import pandas as pd
from keras.models import Sequential
from keras.layers import Dense
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_excel('pima-indians-diabetes (1).xlsx')
X, y = df.iloc[:, :-1], df.iloc[:, -1]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=0)

# Define model
model = Sequential([
    Dense(8, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=20, batch_size=100, validation_data=(X_test, y_test), verbose=1)

# Plot loss
plt.plot(history.history['loss'], 'r--', label='Train Loss')
plt.plot(history.history['val_loss'], 'b-', label='Test Loss')
plt.legend(), plt.xlabel('Epochs'), plt.ylabel('Loss')
plt.show()
```

```

# Accuracy
train_acc = model.evaluate(X_train, y_train, verbose=0)[1] * 100
test_acc = model.evaluate(X_test, y_test, verbose=0)[1] * 100
print(f"Train Accuracy: {train_acc:.2f}% | Test Accuracy: {test_acc:.2f}%")

# Sample predictions
preds = model.predict(X_test[:10])
print("Sample Predictions:\n", preds)

# Plot accuracy
plt.plot(history.history['accuracy'], 'r--', label='Train Acc')
plt.plot(history.history['val_accuracy'], 'b-', label='Test Acc')
plt.legend(), plt.xlabel('Epochs'), plt.ylabel('Accuracy')
plt.show()

```

Epoch 1/20

C:\Users\LENOVO\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:92: Us
When using Sequential models, prefer using an `Input(shape)` object as the first
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

6/6 ————— 5s 185ms/step - accuracy: 0.6316 - loss: 8.7176 - val_acc

Epoch 2/20

6/6 ————— 0s 46ms/step - accuracy: 0.6335 - loss: 7.0879 - val_acc

Epoch 3/20

6/6 ————— 0s 48ms/step - accuracy: 0.6257 - loss: 5.5774 - val_acc

Epoch 4/20

6/6 ————— 1s 47ms/step - accuracy: 0.6296 - loss: 4.1592 - val_acc

Epoch 5/20

6/6 ————— 0s 47ms/step - accuracy: 0.6335 - loss: 2.9721 - val_acc

Epoch 6/20

6/6 ————— 0s 46ms/step - accuracy: 0.6062 - loss: 2.1241 - val_acc

8 RNN

```
[5]: # RNN for Sentiment Analysis on IMDB Reviews
from keras.models import Sequential
from keras.layers import Embedding, SimpleRNN, Dense
from keras.datasets import imdb
from keras.preprocessing import sequence
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
import numpy as np

# Load & preprocess data
max_features, maxlen = 10000, 500
print("Loading data...")
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)
print(f"Train: {x_train.shape}, Test: {x_test.shape}")

# Build model
model = Sequential([
    Embedding(max_features, 32),
    SimpleRNN(32),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='rmsprop', loss='binary_crossentropy', metrics=['acc'])
model.summary()

# Train model
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_split=0.2)
```

```

# Train model
history = model.fit(x_train, y_train, epochs=10, batch_size=128, validation_split=0.2)

# Predict & evaluate
pred = (model.predict(x_test) > 0.5).astype("int32").flatten()
print(classification_report(y_test, pred, target_names=["Negative", "Positive"]))

# Plot accuracy & loss
plt.plot(history.history['acc'], 'bo--', label='Train Acc')
plt.plot(history.history['val_acc'], 'r-', label='Val Acc')
plt.title('Training & Validation Accuracy')
plt.legend(), plt.show()

plt.plot(history.history['loss'], 'bo--', label='Train Loss')
plt.plot(history.history['val_loss'], 'r-', label='Val Loss')
plt.title('Training & Validation Loss')
plt.legend(), plt.show()

```

Loading data...

Train: (25000, 500), Test: (25000, 500)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)